Weitere optische Täuschungen

Du hast schon die sog. Kaffeehaus-Täuschung kennen gelernt und selbst programmiert. Es gibt noch weitere optische Täuschungen, die das Gefühl vermitteln, dass parallele Geraden krumm verlaufen.

Beispiel 1:

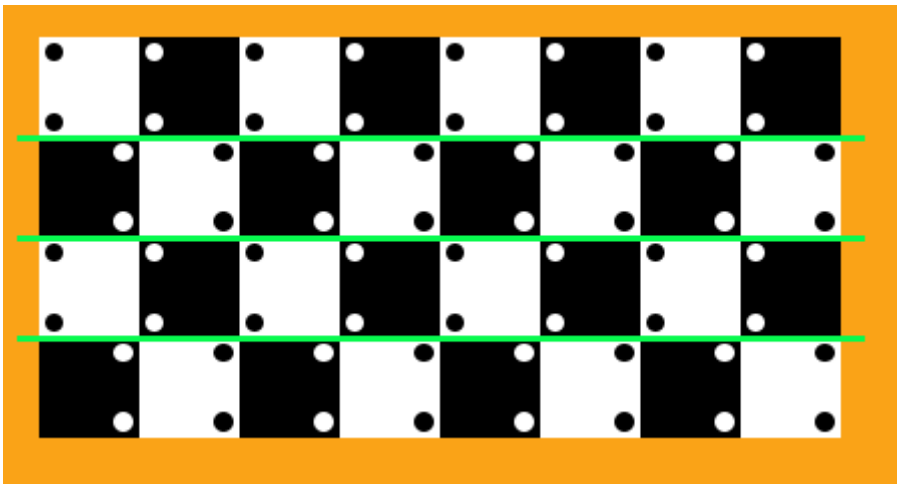
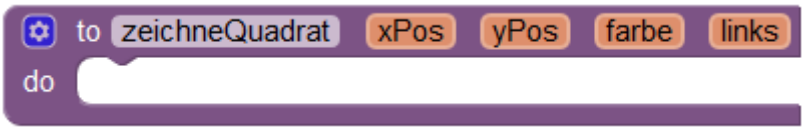


Bild: Screenshot von Ausführung des Programms „wellen1_mit_farbwechsel“ (Eisenmann)

Das Zeichnen des Bildes wäre mit dem, was du bisher in Java gelernt hast, einfach zu programmieren, wenn die kleinen Kreise in den Quadraten nicht wären. Auch diese könnten wir mit Schleifen extra zeichnen lassen, das wäre aber ziemlich umständlich.

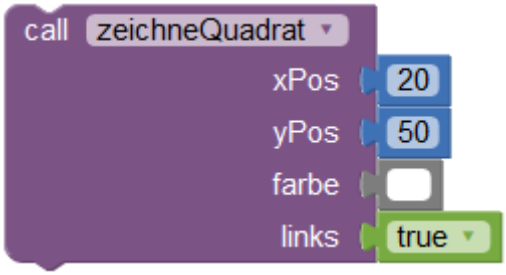
In Scratch oder dem MIT-App-Inventor hast du **Unterprogramme** kennen gelernt. Wir haben Codeteile, die wir häufiger gebraucht haben, zusammengefasst.

Zur Erinnerung:



Das Unterprogramm hier heißt „zeichneQuadrat“ und es können ihm vier Werte übergeben werden. Die Koordinaten der linken oberen Ecke, die Farbe und die Information, ob die Kreise links (true) oder rechts (false) liegen.

Der Aufruf des Unterprogramms könnte z.B. folgendermaßen aussehen:



Unterprogramme gibt es auch in Java. Man nennt sie auch **Methoden**. Wir haben schon einige fertige Methoden genutzt: `ellipse`, `rect`, `fill`, usw. Außerdem haben wir schon einfache Methoden selbst geschrieben.

Um eine Methode zu schreiben, müssen wir uns zunächst überlegen, ob einfach etwas ausgeführt werden soll, oder ob wir etwas ausführen lassen und einen Rückgabewert erwarten.

Allgemein sieht der Rahmen einer **Methode** folgendermaßen aus:

```
<Typ Rückgabewert> <Name der Methode> (<mögliche Übergabeparameter>)
{

 … }
```

Wird keine Rückgabe erwartet, steht vor dem Namen der Methode `void`.

Gibt es keine Übergabeparameter, so bleibt die Klammer leer.

1. *Lies dir das Blatt „Infos zu Methoden in Java“ durch, wenn du es noch nicht bei den Übungen zur Zählschleife gemacht hast.*

Bei der optischen Täuschung von oben möchten wir einfach etwas ausführen lassen (das Zeichnen eines Quadrates mit zwei Punkten), es gibt also keinen Rückgabewert. Um ein Quadrat zeichnen zu können, brauchen wir die Position (Koordinaten der linken oberen Ecke), die Hintergrundfarbe und die Information, ob die Kreise links oder rechts liegen.

Die benötigten Typen hast du schon kennen gelernt: `int` für eine ganze Zahl (auch Farben werden durch ganze Zahlen dargestellt) und `boolean` für die Wahrheitswerte `true` und `false`.

Der Rahmen unserer Methode sieht also folgendermaßen aus:

```
void zeichneQuadrat(int x, int y, int farbe, boolean links) {

}
```

(Die Variablen werden in der Entwicklungsumgebung übrigens unterkringelt, weil sie in der Methode noch nicht genutzt wurden.)

2. *Speichere eine neue Datei unter „wellen1“ ab.*
3. *Schreibe dann zunächst den Rahmen der setup()-Methode.*
4. *Schreibe den Rahmen der eigenen Methode „zeichneQuadrat“ (s.o.).*

Noch sind beide Methoden leer.

5. *Überlege dir mit deiner Sitznachbarin / deinem Sitznachbarn, was in der Methode „zeichneQuadrat“ passieren soll.*

Habt ihr es herausgefunden? Dann holt euch das nächste Blatt.

Es soll ein Quadrat an der vorgegebenen Position mit der vorgegebenen Farbe (weiß oder schwarz) gezeichnet werden.

In dem Quadrat sollen zwei kleine Kreise in der jeweils anderen Farbe entweder links oder rechts gezeichnet werden. Je nachdem, ob `true` (für links) oder `false` (für rechts) übergeben wird.

Um ein Quadrat zeichnen zu können, fehlt uns noch die Seitenlänge.

6. *Deklariert und initialisiert eine globale Variable s für die Seitenlänge.*

Euer Code sollte bisher etwa so aussehen:

```
// globale Variable
int s = 40; // Seitenlänge der Quadrate

void zeichneQuadrat(int x, int y, int farbe, boolean links) {

}

void setup() {

}
```

Bevor wir in der Methode zeichneQuadrat() ein Quadrat zeichnen lassen, müssen wir die Füllfarbe festlegen.

Dann wird die Methode für das Zeichnen eines Rechtecks mit den nötigen Werten aufgerufen.

```
void zeichneQuadrat(int x, int y, int farbe, boolean links) {
  fill(farbe); // der übergebene Wert der Variable farbe wird als Füllfarbe übergeben
  rect(x, y, s, s); // ein Quadrat mit Seitenlänge s wird an der Position (x|y) gezeichnet
}
```

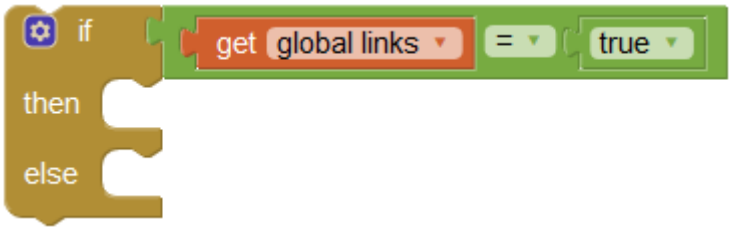
7. *Ergänze dein Programm entsprechend.*
   Du hast gelernt, eine Methode, die du selbst geschrieben hast, in setup-Methode aufzurufen, um sie zu testen. Das kannst Du hier beispielsweise mit dem Befehl zeichneQuadrat(50,100,255,true);
   tun. Hierbei zeichnest du ein Rechteck mit der linken oberen Ecke (50|100) mit der Füllfarbe weiss (255). Das „true“ wird an dieser Stelle noch nicht verwendet.

Jetzt fehlen noch die beiden kleinen Kreise auf der linken oder rechten Seite. Auf welcher Seite sie gezeichnet werden sollen, legt der Übergabeparameter `links` fest. Beim Aufruf der Methode wird entweder `true` oder `false` übergeben. Welche Farbe sie haben, hängt von der Farbe des Quadrates ab.

8. *Ergänze den folgenden Satz für unsere Abbildung: „Falls die Farbe des Quadrates weiß ist, ist die Füllfarbe der Kreise …, sonst … .“*
9. *Ergänze den folgenden Satz: „Falls die Variable `links` den Wert `true` besitzt, dann …, sonst … .“*
10. *Welche Kontrollstruktur brauchst du hier jeweils?*

Wir brauchen eine **Verzweigung**.

Zur Erinnerung:



Falls die Bedingung erfüllt ist (hier: Variable `links` besitzt den Wahrheitswert `true`), wird ausgeführt, was hinter `then` steht, falls nicht, das was hinter `else` steht.

Eine **Verzweigung** in Java sieht folgendermaßen aus:

```
if (<Bedingung>) {
  …
} else {
  …
}
```

In unserem Fall:

```
if (farbe == 255) {
  // Quadrat ist weiß, Füllfarbe muss schwarz werden
} else {
  // Quadrat ist nicht weiß (schwarz), Füllfarbe muss weiß werden
}
```

oder:

```
if (links == true){
  // zeichne die beiden Kreise links
} else {
  // zeichne die beiden Kreise rechts
}
```

Achtung: Für eine Zuweisung genügt ein einfaches Gleichheitszeichen. Möchte man in der Bedingung aber auf Gleichheit überprüfen wie hier, so muss man zwei Gleichheitszeichen schreiben.

11. *Schreibe zunächst in deine Methode zeichneQuadrat(…) die erste Verzweigung zur Farbauswahl der kleinen Kreise. Vergleiche, wenn nötig, mit der ausliegenden Lösung.*
12. *Für die zweite Verzweigung, die sich um das Zeichnen der Kreise kümmern soll, ist eine Skizze (auf Papier) sehr hilfreich. Fertige eine solche Skizze eines Quadrates mit zwei kleinen Kreisen innerhalb an. Überlege dir dabei, welche Werte die Methode ellipse() jeweils übergeben bekommen muss. Nutze des Modus ellipseMode(CORNERS)[1].*

Weitere Aufträge findest du auf dem nächsten Blatt.

1 ellipse(x1,y1,x2,y2) – (x1|y1) linke obere u. (x2|y2) rechte untere Ecke des umschließenden Rechtecks

13. *Schreibe jetzt mithilfe deiner Skizze die Verzweigung, in der die beiden kleinen Kreise gezeichnet werden. (In der vorliegenden Lösung wurde als Kreisdurchmesser ein Viertel der Seitenlänge des Quadrates gewählt und der Einfachheit halber wurden die Kreise zunächst direkt an den Quadratrand gesetzt.)*
14. *Teste dein Programm, indem du in der setup()-Methode zunächst die Größe des Fensters (z.B. 600 auf 400 Pixel) einstellst und dann die Methode „zeichneQuadrat()“ mit verschiedenen Werten aufrufst.*
15. *Vergleiche deinen Programmcode mit der ausliegenden Lösung.*

16. *Schreibe jetzt den Rahmen der Methode alleQuadrate(), in der dann die optische Täuschung gezeichnet werden soll.*
17. *Nutze eine Zählschleife, um die erste Reihe von Quadraten zu zeichnen. Achte darauf, dass von Quadrat zu Quadrat die Füllfarbe wechselt, die Position der Kreise aber gleich bleibt. Nutze dazu eine Variable* `aktuellefarbe` *vom Typ* `int`*.*
18. *Programmiere auch das Zeichnen der zweiten, dritten und vierten Reihe.*
19. *Vergleiche mit der ausliegenden Lösung.*

20. ***** Hast du eine Idee, wie wir **alle** Reihen mithilfe einer verschachtelten Zählschleife zeichnen lassen können? Besprich dich mit deiner Sitznachbarin / deinem Sitznachbarn und probiert eure Ideen aus.*

Weitere Aufgaben:

21. *Programmiert das Zeichnen der grünen Trennlinien zwischen den Reihen.*
22. *Führt eine Variable a für den Abstand der Kreise zum Quadratrand ein und sucht einen Wert, für den die optische Täuschung am wirkungsvollsten ist.*
23. *Verändert euer Programm so, dass die optische Täuschung Abstand zu den Rändern hat.*
24. *Wir haben mehrere Male den Wechsel der Farbe von schwarz auf weiß und umgekehrt benutzt. Schreibe eine Methode „farbwechsel“, der man die Farbe schwarz (0) oder weiß (255) übergeben kann und die die jeweils andere Farbe zurückgibt.*
25. *Nutze die Methode überall dort in deinem Programm, wo ein Farbwechsel stattfindet.*
26. *Teste dein Programm und vergleiche dann mit der ausliegenden Lösung.*

Der MIT App Inventor (*http://appinventor.mit.edu*) wurde ursprünglich von einem Entwicklerteam um Mark Friedman und Hal Abelson bei Google entwickelt und 2012 an das MIT übergeben.

Der MIT App Inventor wird unter der Creative Commons Attribution-ShareAlike 3.0 Unported License veröffentlicht: *https://creativecommons.org/licenses/by-sa/3.0*